Lec-44

21/06/26 4:47 PM

Bit-Masking

0/1

&& → logical and
& → Bit-and
|| → logical or
| → Bit or

! logic not
~

5^5
101
101
000

[2, 3, 7, 3, 2]

^ xor

~(1) = 0

a	b	a&b	a|b	a^b
1	1	1	1	0
0	1	0	1	1
1	0	0	1	1
0	0	0	0	0

a^0 = a

a^b^c = b^c^a = a^c^b

a^a = 0

$2^3 \; 2^2 \; 2^1 \; 2^0$

if (n&1 == 0) {

}

n&1

if ((n&1) == 0) {

}

Left shift (<<)

5 << 2 → 5×2^2 → 20

5 << 3 → 5×2^3

a << n → $a \times 2^n$

Right shift

n = 22

n >> 2 → $\frac{22}{2^2} = \frac{22}{4} = 5$

a >> N = $\boxed{\frac{a}{2^N}}$

-5

8 0101010 0
43210

n = (n >> i)

(n & 1) == 0

1 << 4

10 << 1
100 << 1
1000 << 1
10000

0000 0101
81

A ~~magic~~ number is defined as a number which can be expressed as a power of 5 or sum of unique powers of 5. First few magic numbers are 5, 25, 30(5 + 25), 125, 130(125 + 5), ….

Write a function to find the nth Magic number.

Example:

```
Input: n = 2
Output: 25
Input: n = 5
Output: 130
```

1	2	3	4	5	6	7	8
5	25	30	125	130	150	155	625
5^1	5^2	$5^2 5^1$	5^3	$5^3 5^1$	$5^3 5^2$	$5^3 5^2 5^1$	5^4

i = 8

mul = 1

while (i > 0) {
rem = i % 2
sum = sum + rem * mul
mul = mul * 5
i = i / 2
}

while (i > 0) {
if ((i & 1) != 0) {
sum = sum + mul
}
mul = mul * 5
i >> 1
}

4 7 44 47 74 77 444 447 474 477 744 747 774 777
1 2 3 4 5 6 7 8 9 10

474744

$2 + 2^2 + 2^3 + \dots + 2^{n-1}$

$2\left[\frac{2^{n-1} - 1}{2 - 1}\right] = 2^N - 2$

(1 << N) - 2

474774

$x = 2^6 - 1 = 62$

84 + 1 = 85

abc → 2^n

000 → _
001 → a
010 → b
011 → ab
100 → c
101 → ac
110 → bc
111 → abc

N = 3 limit → 3

```java
public static void print(String s) {
    int n = s.length();
    for (int i = 0; i < (1 << n); i++) {
        Subsequnce(s, i);

    }
}

private static void Subsequnce(String s, int i) {
    // TODO Auto-generated method stub

}
```

i = 5

ans = ac

```java
private static void Subsequnce(String s, int i) {
        // TODO Auto-generated method stub
        String ans = "";
        int pos = 0;
        while (i > 0) {
            if ((i & 1) != 0) {
                ans = ans + s.charAt(pos);

            }
            pos++;
            i >>= 1;
        }
        System.out.println(ans);

    }
```

ans = ac

[5, 6, 3, 2, 12, 16, 14, 5, 6, 14, 3, 2]

n1	n2
5	3
6	2
12	11
14	3
5	2
6	